

Zadanie 16

Natalia Włódyka

Styczeń 11, 2026

1 Polecenie zadania

Celem zadania jest rozwiązanie poniższego układu równań:

$$\begin{aligned}2x^2 + y^2 &= 2 \\ (x - \frac{1}{2})^2 + (y - 1)^2 &= \frac{1}{4}\end{aligned}$$

2 Metoda Newtona dla układu równań wielu zmiennych

Do rozwiązania powyższego układu stosujemy metodę Newtona dla układów równań. Wybieramy początkowy wektor zerowy $x_0 = 0, y_0 = 0$. Iteracyjnie przekształcamy wektory, zgodnie z poniższym równaniem macierzowym:

$$x_{n+1} = x_n - J_F(x_n)^{-1} F(x_n)$$

Gdzie $F(x_n)$ to macierzy zadanego układu równań, a $J_F(x_n)$ to jacobian układu o postaci:

$$J_F(x_n) = \begin{bmatrix} 4x_n & 2y_n \\ 2x_n - 1 & 2y_n - 2 \end{bmatrix} \quad (1)$$

W poniższym kodzie implementujemy ten wzór w postaci układu równań:

$$J_F(x_n)d = F(x_n)$$

A następnie wyznaczamy kolejne przybliżenie:

$$x_{n+1} = x_n - d$$

Kryterium stopu dla metody Newtona to:

$$\|x_{n+1} - x_n\| < \epsilon, \quad \|y_{n+1} - y_n\| < \epsilon$$

3 Kod w Pythonie

```
1 import numpy as np
2 from numpy.typing import NDArray
3
4 numb = np.float64
5
6 def f_x(x: numb, y: numb) -> numb:
7     return 2 * (x**2) + y**2 - 2
8
9 def g_x(x: numb, y: numb) -> numb:
10    return (x - 1/2)**2 + (y - 1)**2 - 1/4
11
12 def jacobian(x: numb, y: numb) -> NDArray[numb]:
13    return np.array([[4*x, 2*y], [2*x - 1, 2*y - 2]], dtype=numb)
14
15 def newton_method(x : numb, y:numb) -> tuple[numb, numb]:
16    tolerance = 1e-6
17    max_iteration = 100
18
19    for i in range(max_iteration):
20        F = np.array([f_x(x, y) , g_x(x, y)])
21        J = jacobian(x, y)
22        d = np.linalg.solve(J, F)
23        new_x = x - d[0]
24        new_y = y - d[1]
25
26        if np.linalg.norm(new_x - x) < tolerance and np.linalg.norm
27            (new_y - y) < tolerance:
28            print(f"Znalezione x = {new_x} i y = {new_y} w
29                iteracjach {i + 1}")
30            return new_x, new_y
31
32        x = new_x
33        y = new_y
34
35    return x, y
36
37 def main():
38    guesses = np.array([[0, 0.5],[0, 1.5],[1, 0.5],[1, 1.5]])
39
40    for guess in guesses:
41        root_x, root_y = newton_method(guess[0], guess[1])
42
43 if __name__ == "__main__":
44    main()
```

4 Rozwiązania układu równań

Rozwiązania zadanego układu równań wynoszą:

$$(x_0, y_0) = (0.1863989268, 1.3894282565), \text{ iteracje} = 8$$
$$(x_1, y_1) = (0.8791207008, 0.6740130461), \text{ iteracje} = 4$$